Software Engineering Lab

Lab 16

Dependency Injection & Inversion of Control

Memisahkan Konfigurasi dari Penggunaan untuk Pengujian Terisolasi di Go



Lukman / lukman-ss

Instansiasi Langsung di Logika Bisnis Mengunci Dependensi

Ketika class memanggil new concrete service di dalam method, sistem terkunci pada infrastruktur.

DAMPAK INSTANSIASI LANGSUNG DI LOGIKA BISNIS

Test Lambat & Rapuh

Pengujian terpaksa menyentuh jaringan atau database eksternal karena dependensi tidak dapat diganti.

DAMPAK INSTANSIASI LANGSUNG DI LOGIKA BISNIS

Vendor Lock-in Kaku

Mengganti gateway pembayaran memaksa refaktorisasi menyeluruh di seluruh lapisan logika domain.

DAMPAK INSTANSIASI LANGSUNG DI LOGIKA BISNIS

Bukan Berarti Rusak

Kode berjalan normal di production, tetapi bukan berarti arsitektur mudah berevolusi tanpa rewrite.

Ketika kelas menginstansiasi dependensi secara langsung di dalam method, logika bisnis terkunci ke infrastruktur konkret. Kode mungkin bekerja tanpa error di tahap awal, bukan berarti arsitekturnya sehat ketika vendor pihak ketiga harus diganti.

Direct object instantiation inside business logic locks domain code to external infrastructure.

content/02-master-draft.md:3-12

Service Locator Terlihat Bersih Namun Menyembunyikan Keterikatan

Menyuntikkan generic container memindahkan instansiasi keluar tetapi mengaburkan kontrak dependensi.

PENDEKATAN NAIF VS REALITA TEKNIS

Kontrak Tersembunyi

Signature NewBadProcessor(Container) menyembunyikan service apa saja yang sebenarnya dibutuhkan.

PENDEKATAN NAIF VS REALITA TEKNIS

Mocking Menjadi Rumit

Unit test wajib mem-mock container dan method lookup alih-alih langsung menyuplai service mock.

PENDEKATAN NAIF VS REALITA TEKNIS

Tidak Otomatis Aman

Menghilangkan kata kunci 'new' tidak otomatis membuat kode loosely coupled jika container bocor ke domain.

Banyak tim mengira Service Locator menyelesaikan kopling karena objek tidak lagi memanggil new secara langsung. Faktanya, menyuntikkan container tidak otomatis menghasilkan kode decoupled jika dependensi sebenarnya tersembunyi di balik API lookup.

Passing a container into business logic hides true dependencies behind runtime lookups.

Pemisahan Konfigurasi dari Penggunaan: Dua Fase Terisolasi

Konstruksi graf objek didelegasikan ke composition root; consumer hanya menerima kontrak.

WIRING ENGINE

1. Composition Root

Hanya main.go atau bootstrap yang berhak membuat struct konkret RealGateway.

BOUNDARY GUARD

2. Interface Contract

Processor hanya mengenal PaymentGateway interface tanpa tahu implementasi jaringan.

STATE VERIFIER

3. Constructor Injection

NewProcessor(g) menjamin dependensi valid saat inisialisasi tanpa mutasi runtime.

DIRECT INSTANTIATION

4. Value Objects

Money dibuat langsung; hanya pada jalur ini objek tanpa IO eksternal tidak di-inject.

Prinsip utama Fowler membedakan dua fase: perakitan graf objek di composition root dan eksekusi logika di domain. Hanya pada jalur ini komponen bisnis tetap bersih dari detail infrastruktur, sementara value object tetap diinstansiasi langsung.

Separate configuration from use: assemble object graphs externally, pass dependencies via interfaces.

Alur Eksekusi: Composition Root ke Processor dan Gateway

Dari wiring di main.go hingga verifikasi input dan delegasi transaksi payment.

MAIN.GO

1. Instantiation

Menginstansiasi RealGateway{} dan menginjeksi ke NewProcessor(realGateway).

PROCESSOR GUARD

2. Input Validation

Memeriksa amount <= 0; mengembalikan error instan sebelum menyentuh gateway.

DOMAIN LAYER

3. Value Assembly

Membentuk struct Money{Amount, Currency} langsung di stack memory.

PAYMENTGATEWAY

4. Gateway Charge

Mendelegasikan Charge(m); belum membuktikan performa di jaringan latency tinggi.

Alur eksekusi dimulai dari instansiasi konkret di composition root, dilanjutkan validasi input lokal sebelum memanggil antarmuka gateway. Arsitektur ini mengisolasi logika, namun belum membuktikan keandalan sistem pada jaringan berlatensi tinggi.

Validate inputs locally at domain boundaries before delegating to external interface contracts.

Constructor Injection: Dependensi Eksplisit dan Valid Saat Lahir

Menerima PaymentGateway via fungsi konstruktor NewProcessor tanpa dependensi tersembunyi.

```
type Processor struct {
    gateway PaymentGateway
}
func NewProcessor(g PaymentGateway) *Processor {
    return &Processor{gateway: g}
}
func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Processor mendeklarasikan gateway sebagai unexported field dan menerimanya saat konstruksi. Penggunaan interface bukan berarti arsitektur bebas alokasi memori; teknik ini menukar overhead pointer interface demi isolasi total dependensi domain.

Constructor injection guarantees immutable references and makes all dependencies explicit in the constructor signature.

internal/di/processor.go:6–21

Kegagalan Service Locator: Keterikatan Tersembunyi Container

Bagaimana NewBadProcessor memecahkan modularitas dan menyulitkan pelacakan dependensi.

SKENARIO KEGAGALAN SERVICE LOCATOR

Dependensi Gelap

Membaca signature konstruktor tidak memberi tahu reviewer bahwa kelas membutuhkan PaymentGateway.

SKENARIO KEGAGALAN SERVICE LOCATOR

Runtime Resolution Bug

Jika container mengembalikan nil saat GetPaymentGateway(), terjadi nil-pointer panic saat runtime.

SKENARIO KEGAGALAN SERVICE LOCATOR

Pelanggaran PSR-11

Passing container ke dalam domain object melanggar panduan PSR-11: 'SHOULD NOT pass container'.

SKENARIO KEGAGALAN SERVICE LOCATOR

Rigid Coupling

Kelas terikat mati pada kontrak Container; hanya pada jalur ini pengujian terpaksa membuat mock container.

Service Locator menyembunyikan dependensi nyata di dalam tubuh method dan berisiko memicu panic saat lookup gagal. Hanya pada jalur ini unit test dipaksa membuat MockContainer tambahan yang tidak relevan dengan kebutuhan bisnis.

Hiding dependencies inside a container transforms compile-time missing dependency checks into runtime failures.

internal/di/locator.go:6–25

Matriks Komparasi: Constructor Injection vs Service Locator

Membedah perbedaan mendasar transparansi kontrak, pengujian, dan siklus hidup komponen.

Dimensi Evaluasi	Constructor Injection (Processor)	Service Locator (BadProcessor)
Deklarasi Dependensi	Eksplisit di parameter NewProcessor	Tersembunyi di balik Container
Type Safety & Validitas	Compile-time check, valid at birth	Runtime resolution, rawan nil panic
Beban Mocking Pengujian	Minimal: langsung injeksi MockGateway	Ganda: wajib MockContainer + MockGateway
Kopling Domain Logic	Nol ke framework / container	Terikat langsung ke interface Container
Rekomendasi Standar	Best practice industri modern	Discouraged (PSR-11: SHOULD NOT)

Matriks ini memetakan keunggulan mutlak Constructor Injection atas Service Locator. Injeksi via konstruktor menjamin verifikasi tipe pada saat kompilasi, sedangkan Service Locator menunda deteksi dependensi yang hilang ke runtime.

Prefer compile-time safety and explicit dependencies over opaque runtime container lookups.

Keputusan Arsitektur: Memilih Pola Perakitan Dependensi

Tiga pertimbangan terstruktur dalam menentukan strategi injeksi dependensi di Go.

SEKUENS KEPUTUSAN WIRING DEPENDENSI

1. Constructor Default

Gunakan Constructor Injection untuk semua service inti demi validitas status dan immutability.

SEKUENS KEPUTUSAN WIRING DEPENDENSI

2. Bypass Value Objects

Instansiasi langsung struct data murni seperti Money; jangan kelola DTO di dalam injector.

SEKUENS KEPUTUSAN WIRING DEPENDENSI

3. Manual Composition

Gunakan manual wiring di main.go; bukan berarti anti-framework, melainkan prioritas kesederhanaan YAGNI.

Keputusan arsitektural menetapkan Constructor Injection sebagai standar baku dan menginstansiasi value object secara langsung. Memilih manual wiring di main.go bukan berarti anti-framework, melainkan keputusan pragmatis menjaga kesederhanaan aplikasi Go.

Default to constructor injection for services, instantiate value objects directly, and assemble graphs in main.

Eksekusi Demo cmd/demo dan Test Suite Lolos Bersih

Output terminal membuktikan kedua pola berjalan dan seluruh 6 unit test lulus tanpa warning.

```
=== RUN   TestProcessor_Success
--- PASS: TestProcessor_Success (0.00s)
=== RUN   TestProcessor_GatewayError
--- PASS: TestProcessor_GatewayError (0.00s)
=== RUN   TestProcessor_InvalidAmount
--- PASS: TestProcessor_InvalidAmount (0.00s)
=== RUN   TestBadProcessor_Success / GatewayError / InvalidAmount
--- PASS: TestBadProcessor_* (0.00s)
PASS: ok lab16/tests 1.356s (race detector: clean)
--- Running Constructor Injection: RealGateway charging 100 USD
--- Running Service Locator: RealGateway charging 200 USD
```

Verifikasi eksekusi membuktikan 6 skenario unit test lolos secara deterministik dalam hitungan detik. Eksekusi demo memperlihatkan kedua mekanisme berhasil memproses transaksi pembayaran tanpa race condition.

Executable verification proves functional parity between patterns while exposing architectural divergence.

engineering/03-execution-result.md:8-46

Batasan Bukti Lab: Yang Dibuktikan vs Belum

Menegaskan batasan verifikasi lab terhadap skenario enterprise berskala masif.

BATAS PEMBUKTIAN SISTEM

YANG DIBUKTIKAN

Isolasi unit test via mock, validasi guard amount <= 0, dan propagasi gateway error.

BATAS PEMBUKTIAN SISTEM

YANG TIDAK DIBUKTIKAN

Lifecycle management singleton/scoped/transient dan performa container otomatis.

BATAS PEMBUKTIAN SISTEM

BATAS KLAIM EMPIRIS

Lab belum membuktikan klaim reduksi defect rate production atau batas baku 12 parameter.

Lab ini membuktikan isolasi pengujian unit dan validasi input sebelum pemanggilan infrastruktur eksternal. Hasil ini belum membuktikan efektivitas lifecycle management kompleks atau klaim penurunan tingkat cacat di production.

Clearly delimit verified architectural properties from unproven enterprise heuristics.

content/01-content-brief.md:12-17

Trade-off Penerapan Dependency Injection

Biaya indireksi dan konfigurasi tambahan berbanding lurus dengan fleksibilitas sistem.

KOMPROMI REKAYASA DEPENDENCY INJECTION

Indirection & Tracing

Pemisahan konstruksi menyulitkan penelusuran implementasi konkret saat membaca kode tanpa IDE canggih.

KOMPROMI REKAYASA DEPENDENCY INJECTION

Boilerplate Inisialisasi

Manual wiring menambah baris perakitan di composition root seiring membesarnya graph objek.

KOMPROMI REKAYASA DEPENDENCY INJECTION

Tidak Otomatis Sederhana

Menerapkan DI tidak otomatis menyederhanakan kode jika kelas belum memiliki abstraksi yang stabil.

Penerapan DI membawa konsekuensi penelusuran alur yang lebih rumit karena implementasi konkret terpisah dari titik penggunaan. Menambahkan interface dan injector tidak otomatis membuat arsitektur lebih baik jika sistemnya masih monolitik sederhana.

DI trades code colocation and simple tracing for loose coupling and isolated testability.

Keamanan Konkurensi pada Komponen Stateless

Processor yang hanya membaca interface gateway aman dieksekusi secara konkuren.

```
// Processor stateless: gateway bersifat read-only setelah konstruksi
type Processor struct {
    gateway PaymentGateway
}
// Aman dipanggil lintas goroutine selama gateway thread-safe
func (p *Processor) ProcessPayment(amount int) error {
    if amount <= 0 { return errors.New("invalid amount") }
    m := Money{Amount: amount, Currency: "USD"}
    return p.gateway.Charge(m)
}
```

Processor bersifat stateless setelah dikonstruksi sehingga aman dipanggil konkuren lintas goroutine tanpa mutasi internal. Simulasi ini tidak sama dengan gateway jaringan riil yang mengelola koneksi socket dan state pool secara terdistribusi.

Stateless service components receiving read-only interface references are inherently thread-safe.

tests/processor_test.go:23-59

Transformasi Desain: Hardcoded vs Constructor Injection

Pergeseran dari kopling ketat infrastruktur ke pemisahan tanggung jawab yang modular.

SEBELUM (HARDCODED) VS SESUDAH (DI VIA CONSTRUCTOR)

Sebelum: new RealGateway()

Logika pembayaran menginstansiasi network client di dalam method; unit test mustahil tanpa internet.

SEBELUM (HARDCODED) VS SESUDAH (DI VIA CONSTRUCTOR)

Sesudah: Constructor Injection

Processor menerima PaymentGateway; unit test berjalan 0.00s menggunakan MockGateway.

SEBELUM (HARDCODED) VS SESUDAH (DI VIA CONSTRUCTOR)

Bukan Berarti Rumit

Refaktorisasi ke constructor injection hanya menambah 3 baris kode namun membuka kemampuan testability.

Sebelum menerapkan DI, logika pembayaran terkunci pada implementasi RealGateway konkret yang menyulitkan pengujian offline. Memisahkan konstruksi ke NewProcessor bukan berarti menambah kompleksitas, melainkan membebaskan domain dari detail eksternal.

Extract concrete instantiation to constructor parameters to transform flaky integration tests into instant unit tests.

Interface Minimal dan Pengecualian untuk Value Object

Mendefinisikan kontrak abstraksi ramping serta aturan kapan DI tidak boleh digunakan.

```
// Money adalah value object data murni: instansiasi langsung
type Money struct {
    Amount   int
    Currency string
}
// PaymentGateway mendefinisikan kontrak minimal abstraction
type PaymentGateway interface {
    Charge(m Money) error
}
```

Interface PaymentGateway dirancang ramping hanya dengan satu method Charge, sedangkan Money diinstansiasi langsung tanpa perantara container. Keputusan membedakan service dan value object ini cakupannya mengikuti test skenario spesifik lab ini.

Inject external infrastructure services; instantiate pure immutable value objects directly.

internal/di/gateway.go:5–15

Pengujian Terisolasi Menggunakan Test Double MockGateway

Menguji logika Processor tanpa menyentuh koneksi jaringan riil dengan determinisme mutlak.

```
type MockGateway struct {
    ChargedMoney di.Money
    ShouldFail   bool
}
func (m *MockGateway) Charge(money di.Money) error {
    if m.ShouldFail { return errors.New("gateway unavailable") }
    m.ChargedMoney = money
    return nil
}
```

MockGateway mensimulasikan kegagalan jaringan dan mencatat payload transaksi tanpa koneksi HTTP riil. Menulis unit test dengan mock tidak otomatis menjamin integritas integrasi vendor jika payload skema production mengalami perubahan.

Test doubles verify business logic deterministically without incurring network latency or side effects.

tests/processor_test.go:10–21

Validasi Pre-Check Mencegah Panggilan Jaringan Sia-sia

Verifikasi amount <= 0 memastikan gateway tidak pernah dipanggil pada payload invalid.

```
func TestProcessor_InvalidAmount(t *testing.T) {
    mock := &MockGateway{}
    proc := di.NewProcessor(mock)
    err := proc.ProcessPayment(-10)
    if err == nil { t.Fatalf("expected error for negative amount") }
    if mock.ChargedMoney.Amount != 0 {
        t.Errorf("gateway should not have been called, got %d",
            mock.ChargedMoney.Amount)
    }
}
```

Test memastikan bahwa input negatif langsung ditolak di lapisan domain sebelum menyentuh gateway. Hanya pada jalur ini mock memverifikasi bahwa amount tetap 0, membuktikan panggilan jaringan berhasil dicegah sejak awal.

Enforce business validation guards before delegating to downstream infrastructure dependencies.

tests/processor_test.go:47-59

Propagasi Error Transparan dari Lapisan Infrastruktur

Ketika gateway eksternal gagal, error diteruskan ke caller tanpa ditelan secara diam-diam.

```
func TestProcessor_GatewayError(t *testing.T) {
    mock := &MockGateway{ShouldFail: true}
    proc := di.NewProcessor(mock)
    err := proc.ProcessPayment(50)
    if err == nil {
        t.Fatalf("expected error from gateway, got nil")
    }
    // Verifikasi error gateway sampai ke pemanggil
}
```

Processor meneruskan error dari gateway secara transparan ke caller. Meneruskan error mentah bukan berarti sistem tidak butuh retry policy; ini membuktikan kontrak interface mampu mengalirkan status kegagalan infrastruktur secara akurat.

Transparent error propagation preserves downstream failure context for upstream callers.

Composition Root di cmd/demo: Satu Titik Perakitan Objek

Inisialisasi komponen konkret dan perakitan graf dilakukan di titik masuk aplikasi.

```
func main() {
    // Instansiasi dependensi konkret
    realGateway := &di.RealGateway{}
    // Constructor Injection eksplisit
    processor := di.NewProcessor(realGateway)
    _ = processor.ProcessPayment(100)
    // Service Locator anti-pattern untuk demonstrasi
    container := &SimpleContainer{gateway: realGateway}
    badProcessor := di.NewBadProcessor(container)
    _ = badProcessor.ProcessPayment(200)
}
```

Fungsi main bertindak sebagai composition root tunggal tempat semua struct konkret dibuat dan dirakit. Struktur sederhana ini sengaja dipilih tanpa framework DI pihak ketiga, dan cakupannya mengikuti test verifikasi perilaku lab.

Centralize concrete component instantiations strictly within the application composition root.

cmd/demo/main.go:16-36

Checklist Penerapan Dependency Injection di Go

Pedoman wajib sebelum menggabungkan pull request yang mengelola dependensi infrastruktur.

- ✓ Dependensi infrastruktur dideklarasikan sebagai interface, bukan struct konkret.
- ✓ Constructor injection digunakan sebagai default; semua dependensi eksplisit di parameter.
- ✓ Composition root (main.go atau wire bootstrap) adalah satu-satunya perakit implementasi konkret.
- ✓ Tidak ada pemanggilan generic container.Get() di dalam lapisan logika domain bisnis.
- ✓ Value object murni (seperti Money) diinstansiasi langsung tanpa melalui injection container.
- ✓ Seluruh service memiliki unit test terisolasi menggunakan mock tanpa menyentuh jaringan.

Apply strict architectural gates to ensure dependencies remain explicit and testable.

Anti-Pattern Umum dalam Penerapan Dependency Injection

Kekeliruan perancangan yang sering merusak modularitas dan keterbacaan kode.

HINDARI KESALAHAN UMUM INI

Service Locator Leak

Mengoper Container ke dalam domain; menyembunyikan dependensi nyata di balik lookup method runtime.

HINDARI KESALAHAN UMUM INI

Over-Injection & Bloat

Konstruktor menerima belasan dependensi; sinyal keras pelanggaran Single Responsibility Principle.

HINDARI KESALAHAN UMUM INI

Value Object Injection

Memasukkan struct data murni seperti Money atau DateTime ke container DI; bukan berarti lebih bersih.

Menyuntikkan container ke domain menyembunyikan dependensi, sedangkan konstruktor dengan belasan parameter menandakan pelanggaran SRP. Memasukkan value object ke dalam container bukan berarti arsitektur lebih rapi, melainkan menambah boilerplate sia-sia.

Avoid container leakage, prevent constructor parameter bloat, and never inject pure value objects.

Reproduksi Mandiri di Mesin Lokal Menggunakan Go

Eksperimen validasi mandiri tanpa dependensi pihak ketiga (zero-dependency).

```
cd labs/16-dependency-injection
go test -v ./...    # validasi unit test terisolasi Processor dan BadProcessor
go test -race ./... # verifikasi konkurensi stateless bebas race condition
go run ./cmd/demo   # eksekusi komparasi Constructor Injection vs Service Locator
```

Seluruh mekanisme validasi ditulis menggunakan Go standard library murni tanpa framework eksternal. Skenario lokal ini memverifikasi isolasi logika, namun production masih perlu mempertimbangkan auto-wiring tool seperti wire pada skala ribuan komponen.

Zero-dependency local verification guarantees fast, repeatable validation across developer workflows.

README.md:13-23

Prinsip Inti: Pisahkan Konfigurasi, Eksplisitkan Kontrak

Arsitektur bertahan lama ketika logika bisnis terbebas dari mekanisme perakitan infrastruktur.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Pisahkan Konfigurasi

Biarkan composition root merakit graf objek; modul bisnis fokus murni pada aturan domain.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Kontrak Eksplisit

Gunakan Constructor Injection agar dependensi terlihat transparan sejak kompilasi.

TIGA PILAR KELANGSUNGAN ARSITEKTUR

Disiplin Batas Domain

Menerapkan DI tidak otomatis menyelesaikan semua masalah jika batasan bounded context tidak dirancang benar.

Dependency Injection adalah teknik memisahkan fase konfigurasi dari eksekusi logika. Menyuntikkan dependensi via interface tidak otomatis menyelesaikan kekacauan arsitektur jika tanggung jawab domain sejak awal tidak dirancang dengan batas yang tegas.

Decouple configuration from execution to preserve long-term evolvability and testability.

Logika bisnis yang menginstansiasi dependensinya sendiri adalah sistem yang menolak untuk diuji.

Business logic that instantiates its own dependencies creates code that resists testing.

Software Engineering Lab 16 – Dependency Injection & Inversion of Control

internal/di/gateway.go • internal/di/processor.go • internal/di/locator.go
cmd/demo/main.go • tests/processor_test.go
engineering/01–design.md • content/02–master-draft.md

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/16-dependency-injection`

Lukman / lukman–ss

Commit: 5563838 • Status: APPROVED
Test passing: 6 unit tests (happy path, gateway error, input validation pre-check)
Race detector: Clean (go test -race ./... PASS in 1.35s)
Manual wiring Go murni; bukan framework DI berbasis refleksi runtime.